นายอติสัญจ์ รัตนะมงคลกุล 6609650228

ตอนที่ 1 (50 คะแนน)

ข้อที่ 1 (ข้อเขียน, 20 คะแนน)

อธิบาย PEAS และ บอกคุณสมบัติของสภาพแวดล้อม (Properties of task environments) ของกิจกรรมต่อไปนี้

1.1 ถักเสื้อไหมพรม
1.2 ฝึกตีเทนนิสกับกำแพง
1.3 แข่งเทนนิสกับผู้เล่นอีกหนึ่งคน

ตัวอย่างของ Properties of task environments

Task Environment	Observable	Agents	Deterministic/ Stochastic	Episodic/ Sequential	Static/ Dynamic	Discrete/ Continuous
Taxi Driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous

อธิบายตามความเข้าใจของนักศึกษาเกี่ยวกับ Terms ต่อไปนี้

Agent, Agent function, Agent program, Rationality, Simple reflex agent, Model-based agent, Goal-based agent, Utility-based agent, learning agent

1.1) ถักเสื้อไหมพรม

Performance : ความเร็วในการถัก , ความสมบูรณ์ของเสื้อ
Environment : โต๊ะทำงาน ,ไหมพรม,ไม้ถัก
Actuators : มือที่ถักเสื้อ
Sensors : สัมผัสจากมือ ,การมองเห็น

Task Environment	Observable	Agents	Deterministic/ Stochastic	Episodic/ Sequential	Static/Dynamic	Discrete/ Continuous
ถักเสื้อไหมพรม	Partially	Single	Deterministic	Sequential	Static	Continuous

1.2) ฝึกตีเทนนิสกับกำแพง

Performance : ความแม่นยำในการตีลูก , จำนวนครั้ง
Environment : คอร์ทเทนนิส , กำแพง, ลูกเทนนิส, ไม้เทนนิส
Actuators : แขนใช้ตีลูก ,ขาใช้เคลื่อนที่
Sensors : ตามองลูก ,หูฟังเสียง

Task Environment	Observable	Agents	Deterministic/ Stochastic	Episodic/ Sequential	Static/Dynamic	Discrete/ Continuous
ฝึกตีเทนนิสกับกำแพง	Fully observable	Single	Deterministic	Sequential	Dynamic	Continuous

1.3) แข่งเทนนิสกับผู้เล่นอีกฝั่งหนึ่ง

Performance : คะแนนที่ได้รับจากการตีให้ฝั่งตรงข้ามรับไม่ได้

Environment : คอร์ทเทนนิส, ตาข่าย, ผู้เล่นฝั่งตรงข้าม, ไม้เทนนิส, ลูกเทนนิส

Actuators : แขนใช้ตีลูก, ขาใช้เคลื่อนที่

Sensors : ตามองลูกและฝั่งตรงข้าม, หูฟังเสียง

Task Environment	Observable	Agents	Deterministic / Stochastic	Episodic / Sequential	Static / Dynamic	Discrete / Continuous
แข่งเทนนิสกับผู้เล่นอีกฝั่งหนึ่ง	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous

Agent : คืออะไรก็ตามที่สามารถรับรู้สภาพแวดล้อมผ่าน Sensors และสามารถแสดงพฤติกรรมต่อสภาพแวดล้อมผ่าน Actuators

Agent function : การกำหนดการกระทำของ Agent ตามการรับรู้จาก Sensors

Agent program : การกำหนดตรรกะของ Agent ในโปรแกรม

Rationality : การเลือกการกระทำที่ดีที่สุดตามข้อมูลที่มีอยู่

Simple reflex agent : Agent ที่ตัดสินใจจากปัจจุบันของสภาพแวดล้อมโดยตรง เป็นประเภทที่ซับซ้อนน้อยที่สุด

Model-based agent : Agent ที่มีประสิทธิภาพในการคาดการณ์ผลลัพธ์ของการกระทำ

Goal-based agent : Agent ที่มีเป้าหมาย (Goal) บางครั้งการกระทำจะตรงไปตรงมา เพราะตอบสนองต่อ Goal

Utility-based agent : Agent ที่เลือกการกระทำโดยพิจารณาผลลัพธ์ให้ออกมาดีที่สุด

Learning agent : Agent ที่สามารถเรียนรู้และปรับปรุงพฤติกรรมจากประสบการณ์

นายวุฒิพัฒน์ วัชระมงคลกุล 6404650228

ข้อที่ 2 (ข้อเขียน + เขียนโปรแกรม, 15 คะแนน)

2.1 ให้นักศึกษา<u>ออกแบบ</u>และ<u>อธิบาย</u>วิธีการทำงานของเครื่องดูดฝุ่นอย่างง่ายที่เป็นแบบ Simple reflex agent (เขียน Agent function) โดยสมมติให้ห้องที่เป็นไปได้มีทั้งหมด 6 ห้องคือ A B C D E F ตามภาพด้านล่าง โดยต้องกำหนด Actions, Rules, States ที่เหมาะสม โดยสถานะที่เป็นได้ในแต่ละห้องคือ (Dirty/Clean)

A [เครื่องดูดฝุ่น]	B	E
C	D	F

2.2 เขียน Agent program เพื่อ Implement agent function ที่นักศึกษาได้ออกแบบไว้แล้วในข้อ 2.1

2.1)

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Right
[B, Dirty]	Suck
[E, Clean]	Down
[E, Dirty]	Suck
[F, Clean]	Left
[F, Dirty]	Suck
[D, Clean]	Left
[D, Dirty]	Suck
[C, Clean]	Up
[C, Dirty]	Suck

2.2)

```
Tabnine | Edit | Test | Explain | Document
char *formulateProblem(struct Room *room, char *goal) {
    if (strcmp(goal, "Clean") == 0) return "Suck";


    if (strcmp(room->name, "A") == 0) return "Right";
    if (strcmp(room->name, "B") == 0) return "Right";
    if (strcmp(room->name, "E") == 0) return "Down";
    if (strcmp(room->name, "F") == 0) return "Left";
    if (strcmp(room->name, "D") == 0) return "Left";
    if (strcmp(room->name, "C") == 0) return "Up";
}
```

นายวุฒิพัฒน์ วัชระมงคลกุล 6404650228

ข้อที่ 3 (ข้อเขียน, 15 คะแนน)

พิจารณาปัญหาหอคอยฮานอย (Towers of Hanoi problem)



ปัญหาหอคอยฮานอยเป็นปัญหาที่มักใช้เป็นตัวอย่างในการศึกษาเกี่ยวกับปัญหาที่เป็น Recursion ในวิทยาการคอมพิวเตอร์ ตัวอย่างเช่น การเขียน Recurrence relation ในคณิตศาสตร์แบบไม่ต่อเนื่อง เริ่มต้นที่แผ่นกลม N แผ่นที่มีขนาดแตกต่างกันถูกวางในแท่น 1 แท่น ในขณะที่อีก 2 แท่นว่างเปล่า

เราสามารถย้ายแผ่นกลมจากแท่นหนึ่งไปยังอีกแท่นหนึ่งได้แต่มีเงื่อนไขว่า **แผ่นกลมที่ใหญ่กว่าจะไม่วางอยู่เหนือแผ่นกลมที่เล็กกว่าเสมอไม่ว่าแท่นใด** และ **เป้าหมายคือต้องการย้ายแผ่นกลมทั้งหมดไปไว้ในแท่นทางด้านขวาสุด**

3.1 ให้นักศึกษาออกแบบและกำหนด problem formulation สำหรับปัญหานี้อย่างละเอียด (State, Initial state, Actions, Transition model, Goal test, Path cost)

3.2 หาขนาด Size ของ State space ในกรณีที่กำหนดให้มีแท่น k แท่น และ แผ่นกลม n แผ่น

3.1) Problem formulation

State : ตำแหน่งของแผ่นกลมแต่ละแผ่น เช่น (แผ่นที่ 1 อยู่ที่ แท่น A , แผ่นที่ 2 อยู่ที่ แท่น B, แผ่นที่ 3 อยู่ที่ แท่น C = (A,B,C))

Initial state: แผ่นกลมทั้งหมดอยู่ในแท่นแรก เช่น แท่น A

Actions: ย้ายแผ่นกลมไปยังแท่น โดยต้องเป็นการเคลื่อนย้ายที่ถูกต้องและอยู่ในเงื่อนไข

Transition model: เปลี่ยนสถานะ จากการย้าย เช่น หากแผ่นเล็กสุดย้าย จากแท่น A ไป B จะได้สถานะใหม่

Goal test: ทุกแผ่นอยู่ที่แท่นสุดท้าย (แท่น C)

Path cost: จำนวน การย้ายแผ่นกลม

3.2) คำนวณจำนวนสถานะทั้งหมด

กำหนด n = จำนวนแผ่นกลม

k = จำนวนแท่น

Size of State Space = k^n

Example

กำหนด $n=3$ $k=3$ → k^n → $3^3 = 27$ "

นายจุติพัฏร์ วัชระมงคลกุล 6404650228



- โปรแกรมนี้ ใช้ลำดับ ค้นหาเส้นทางกราฟ โดยมี 5 Algorithm ดังนี้

1. Depth-First Search (DFS)
2. Breadth_First Search (BFS)
3. Uniform Cost Search (UCS)
4. Greedy Best-First Search (GBFS)
5. A-Search (A*)

คู่มือการรันโปรแกรม

1. Compile and Run โปรแกรม
2. ป้อน Input Initial Node และ Goal Node

```
Enter Initial Node (S, A, B, C, D, F): S
Enter Goal Node (S, A, B, C, D, F): F
Select Algorithm:
1. DFS
2. BFS
3. UCS
4. GBFS
5. A*
Enter choice: 1
DFS Path: S -> A -> B -> D -> F -> Goal reached!
```

Depth-First Search (DFS)

```
Enter Initial Node (S, A, B, C, D, F): S
Enter Goal Node (S, A, B, C, D, F): F
Select Algorithm:
1. DFS
2. BFS
3. UCS
4. GBFS
5. A*
Enter choice: 2
BFS Path: S -> A -> B -> C -> D -> F -> Goal reached!
```

Breadth_First Search (BFS)

```
Enter Initial Node (S, A, B, C, D, F): S
Enter Goal Node (S, A, B, C, D, F): F
Select Algorithm:
1. DFS
2. BFS
3. UCS
4. GBFS
5. A*
Enter choice: 3
UCS Path: S -> A -> B -> D -> C -> F -> Goal reached!
```

Uniform Cost Search (UCS)

```
Enter Initial Node (S, A, B, C, D, F): S
Enter Goal Node (S, A, B, C, D, F): F
Select Algorithm:
1. DFS
2. BFS
3. UCS
4. GBFS
5. A*
Enter choice: 4
Greedy Best-First Search Path: S -> B -> D -> F -> Goal reached!
```

Greedy Best-First Search (GBFS)

```
Enter Initial Node (S, A, B, C, D, F): S
Enter Goal Node (S, A, B, C, D, F): F
Select Algorithm:
1. DFS
2. BFS
3. UCS
4. GBFS
5. A*
Enter choice: 5
A* Search Path: S -> B -> A -> D -> F -> Goal reached!
```

A-Search (A*)